DrainSinkhorn: Safe Elimination for Batched Entropic Optimal Transport*

Xinyang Wen

Abstract

Fast OT backends reduce the cost of one Sinkhorn update, but fixed-width batched execution still runs at full width until its slowest problem finishes. We present DrainSinkhorn, an execution layer for batched entropic optimal transport that dynamically removes completed problems and narrows subsequent updates. It starts from standard batched execution of independent Sinkhorn problems. After an alternating scaling cycle, one marginal is current by construction; a batched screening of the other marginal filters the problems that require the two-marginal verifier. Problems that pass that existing release rule are removed from every per-problem tensor, while unfinished problems continue.

The analysis separates opportunity from hardware conversion. For first-passing iteration numbers $n_1, \ldots, n_W$ and survivor width $A_k = |\{t : n_t \geq k\}|$, static execution spends $Wn_{\max}$ problem-update slots, whereas dynamic compaction spends exactly $\sum_k A_k = \sum_t n_t$. If $c_k(a, \xi_k)$ is the measured update cost at width a, the exact matched-path time difference is the width-cost saving $\sum_k [c_k(W, \xi_k) - c_k(A_k, \xi_k)]$ minus incremental control overhead. Depth heterogeneity therefore creates removable work, while the hardware width-cost curve determines how much of that work becomes wall-clock speedup. On the measured A100/Triton configuration, width sweeps expose a wave-shaped cost curve: for $n = 1024$, widths one through six occupy one nearly flat region, whereas larger problems respond to width reduction much earlier.

Matched fixed-width/dynamic-compaction comparisons give 1.746× on the four-worker MetroPT-3 endpoint, 1.054× on the ImageNet-32 training solver field, and up to 3.110× in the registered MetroPT scale-out cells. An OTT-JAX realization gives 1.366–1.581× across three tolerances. When one initialization is used for multiple problems, matched packed-Triton dynamic compaction gives 1.421× and removes 30.62% of problem-update slots. Packer19 supplies the matched logical-mask control and a synchronized-depth boundary where the dynamic-compaction gain vanishes. Achieved residuals and registered application endpoints supply the implementation evidence; all reported consumer and training-quality checks pass.

*Code: github.com/cis-aconiticacid/DrainSinkhorn

1 Introduction

GPU implementations of entropic optimal transport (EOT) have become markedly better at executing a single Sinkhorn update. Matrix-free operators, tiled reductions, and fused streaming kernels reduce memory traffic and avoid materializing the transport matrix [4, 5]. Many workloads, however, prepare a batch of couplings for predictive maintenance, scientific state transitions, or mini-batch training. Once the inner update is fast, total cost also depends on how long each problem remains in that batch.

Different problems usually reach the stopping tolerance at different iterations. A static batch runs until the problem requiring the most iterations finishes: completed problems continue to occupy state, verification bandwidth, and kernel lanes. A logical mask freezes their state but does not shrink the tensor seen by a fused kernel. The unused rectangle between the iteration number at which each problem passes and the largest such number is therefore a distinct source of OT work.

We introduce DrainSinkhorn, an execution layer that removes this padding. It starts from standard batched execution of independent Sinkhorn problems and removes a problem after it passes verification used by the selected solver. A full alternating cycle makes the newly updated marginal current in exact arithmetic, so the other marginal provides a cheap batched screen. Lanes that pass the subsequent two-marginal verifier are removed from potentials, marginals, support descriptors, and scratch buffers. The next backend update runs at the width of the unfinished set.

This design separates inner-kernel acceleration from cross-problem execution. The inner backend determines the cost of one stabilized update; DrainSinkhorn changes the number and width of updates executed across a batch. We implement the execution layer with a packed Triton kernel, OTT-JAX, and PyKeOps. The packed Triton path additionally uses the current marginal in a preliminary screen; the OTT-JAX and PyKeOps paths remove completed problems with independent backend mechanisms.

For problem t, let n_t be the batched-iteration number of the first configured verification round at which it passes the two-marginal verifier. Fixed-width execution consumes $Wn_{\max}$ problem-update slots and execution with dynamic compaction consumes exactly $\sum_t n_t$. This

task-independent identity depends only on the observed iteration numbers. A probabilistic form sharpens the condition: the expected removable work is positive precisely when a window has positive probability of containing both completed and unfinished lanes before some batched iteration. Removable slots are only the opportunity. If $c_k(a, \xi_k)$ denotes the measured cost of update k at width a, the wall-clock saving is $\sum_k [c_k(W, \xi_k) - c_k(A_k, \xi_k)]$ minus incremental control overhead. The survivor curve and the backend width-cost curve are therefore the two quantities that determine realized acceleration.

GPU width cost can be quantized by scheduling waves rather than proportional to the number of problems. The packed Triton update launches a width-dependent number of thread blocks; reducing A_k saves time when the surviving grid moves to a lower cost level. The solver's verifier supplies the survivor trajectory, and a measured width sweep supplies the hardware conversion curve.

The contribution is evaluated from conventional fixed-width batched execution: DrainSinkhorn adds stopping-rule-triggered release and dynamic compaction while holding the selected inner backend fixed. OTT-JAX and PyKeOps provide independent backend realizations. The experiments distinguish OT-stage acceleration from application endpoints: the former measures the work removed inside EOT, while the latter tests whether that acceleration survives unchanged consumer or model work.

Our contributions are:

1. **Dynamic compaction during batched Sinkhorn.** Starting from standard fixed-width batching and the selected solver's stopping rule, we dynamically remove completed problems from all per-problem state. Each subsequent kernel executes at a width of the unfinished set.

2. **Opportunity and conversion accounting.** A survival-curve identity counts removed problem-update slots. An exact matched-path time decomposition maps the survivor widths through the backend width-cost curve and subtracts incremental control overhead. This separates the instance-side opportunity from its hardware conversion.

3. **Causal component evidence.** Matched controls separate batched verification, Sinkhorn-specific screening, logical masking, and dynamic compaction. Grouping experiments that hold total iteration counts fixed and direct width-cost measurements identify the two conditions under which dynamic compaction pays.

4. **Backend and endpoint evidence.** MetroPT-3, Packer19, and ImageNet-32 evaluate the packed Triton path; OTT-JAX and PyKeOps test transfer on ImageNet-32 and A2D2. The resulting paths improve over the specified references using the same backend on the tested workloads. Numerical behavior is reported through parity, residual, and finite-precision stress tests, while complete consumers test application-visible output and training quality.

Fast kernels make an update cheaper. Dynamic compaction removes updates that would otherwise be issued only to already completed problems.

2 Setting and Prior Systems

Entropic optimal transport. For positive unit-mass marginals $a \in \mathbb{R}^n_+$ and $b \in \mathbb{R}^m_+$, EOT solves

$$\min_{\pi \geq 0} \langle C, \pi \rangle + \varepsilon \sum_{ij} \pi_{ij}(\log \pi_{ij} - 1), \quad \pi \mathbf{1} = a, \quad \pi^\top \mathbf{1} = b. \tag{1}$$

With $K = \exp(-C/\varepsilon)$, Sinkhorn alternates diagonal scalings to form $\pi = \operatorname{diag}(u) K \operatorname{diag}(v)$ [3]. Coordinate, relaxed, low-rank, and Newton methods reduce arithmetic or iteration count inside one solve [1, 8, 13, 14]. Fused streaming implementations rewrite stabilized updates as LogSumExp reductions and reduce memory traffic [17]. DrainSinkhorn keeps the chosen inner update and changes the set of problems that receive the next one.

Initialization and repeated couplings. Carry, analytic extensions, and learned dual proposals can reduce a new problem's starting defect [2, 15]. Large-coupling Flow Matching also uses soft c-transforms when supports change [20]. These mechanisms change the numbers of iterations required by the problems. DrainSinkhorn accommodates both regimes: the formal Packer19 attribution uses the same fixed proposal in every arm, and the ImageNet-32 training experiment uses independent cold-start couplings.

Dynamic batching and batched solvers. Automatic batching tracks the program point of each member in a data-dependent program [12]. Orca schedules autoregressive inference one iteration at a time [19]. Outside OT, Ginkgo's batched CG provides a related execution precedent: each uniform independent linear system exits once its residual reaches the stopping tolerance, and later iterations skip converged systems inside the fused solver [7]. jaxipm redesigns IPOPT around heterogeneous iteration fusion and replacement of completed problems [16]. Within OT, POT's `solve_batch` parallelizes independent OT problems represented by batched cost matrices, and OTT-JAX uses JAX transformations to vectorize multiple Sinkhorn solves [4, 6, 11]. Fixed-width multi-problem Sinkhorn execution is the starting point for our work. We target the subsequent removal of completed EOT problems while retaining online pairwise interactions instead of materializing one cost tensor per problem.

DrainSinkhorn addresses the numerical structure of EOT batches. Alternating scaling permits a preliminary screen using the current marginals; the two-marginal verifier is evaluated on problem outputs; dynamic compaction must preserve the batch-element correspondence of potentials, marginals, supports, and scratch state. Generic masking leaves those EOT-specific state transformations

and the width unchanged. Table 1 compares the batched object, completion signal, and execution action of prior systems. Our OTT-JAX and PyKeOps variants remove completed problems, whereas the packed Triton realization also performs a preliminary screen using the current marginals.

OT consumers. OTT provides a broad transport API [4]. Multisample Flow Matching consumes an OT coupling to select training pairs [10]; these couplings can be prepared independently of network optimization [20]. We evaluate the complete path from the OT coupling to the downstream consumer for industrial endpoints and carry the same execution path through a fixed-update feature-space OT-FM training pipeline.

3 DrainSinkhorn

3.1 Problem and objective

Consider W independent entropic optimal-transport (EOT) problems executed by one batched Sinkhorn backend. Standard fixed-width batching applies the same per-problem Sinkhorn map in parallel but retains all W lanes until the slowest problem satisfies the configured stopping rule. When completion depths differ, already-finished lanes therefore remain in later batched kernels. Our objective is to remove this padding without changing any problem's EOT objective, Sinkhorn update, stopping tolerance, two-marginal release rule, or downstream consumer.

3.2 Method overview

DrainSinkhorn begins from the same fixed-width batch and repeatedly performs a batched update, a preliminary one-marginal screen, the configured two-marginal verification for screen-positive lanes, and physical compaction of every lane that passes. For problem t, let n_t be the batched-iteration number of the first configured verification at which the problem passes. DrainSinkhorn then writes its output and removes all of its per-problem state. Batched update k therefore executes at survivor width

$$A_k = |\{t : n_t \geq k\}| .$$

The verification interval is the number of batched iterations between consecutive verifications. Logical masking records completion at width W; dynamic-compaction execution runs subsequent kernels at survivor width A_k. A fixed-width/dynamic-compaction comparison measures the net effect of release and dynamic width reduction. LM/DC holds the release decisions fixed to isolate dynamic width reduction. The experiments report width and grouping interventions separately.

3.3 Standard batched Sinkhorn execution

Let F_t denote one full Sinkhorn cycle for problem t, including that problem's marginals, kernel operator, and numerical representation. Standard batching adds a leading problem-batch dimension but no cross-problem reduction. In ideal arithmetic, its defining algebraic property is

$$\widetilde{F}(\mathrm{pack}(z_1, \ldots, z_W)) = \mathrm{pack}(F_1(z_1), \ldots, F_W(z_W)) . \quad (2)$$

Thus the batched kernel preserves the EOT objective and per-instance Sinkhorn map while changing their execution layout. After a verification, DrainSinkhorn filters the problem-batch dimension and applies Equation (2) to the survivors.

For one problem, a Sinkhorn cycle performs

$$u^{k+1} = a \oslash (Kv^k), \qquad v^{k+1} = b \oslash (K^\top u^{k+1}), \quad (3)$$

where $\oslash$ denotes elementwise division. Immediately after the second leg, exact arithmetic gives

$$v^{k+1} \odot (K^\top u^{k+1}) = b. \quad (4)$$

The column marginal is current by construction; the row marginal $u^{k+1} \odot (Kv^{k+1})$ can remain outside tolerance. DrainSinkhorn evaluates that row residual for all active problems in one batched operation and sends only screen-positive problems to the two-marginal verifier.

3.4 Screen, verify, and compact

Let $\widehat{r}^{\mathrm{row}}_{t,k}$ be the batched screen value. A problem is screen-positive when $\widehat{r}^{\mathrm{row}}_{t,k} \leq \tau_{\mathrm{screen}}$; otherwise it continues without materializing the more expensive two-marginal verification. For a screen-positive problem, the implementation recomputes both marginals and applies the configured stopping rule

$$r_{t,k} = \max\{\|\pi_{t,k}\mathbf{1} - a_t\|_1, \|\pi_{t,k}^\top \mathbf{1} - b_t\|_1\} \leq \tau_{\mathrm{stop}}. \quad (5)$$

A rejected problem remains active for a later verification. A false-positive screen invokes the two-marginal verifier unnecessarily, whereas a false-negative screen delays removal. We measure both effects by replaying the screen outcomes and evaluating every replayed candidate with the verifier in section 5.

For each passing problem, DrainSinkhorn writes the output and compacts every per-problem object: source and target descriptors, marginals, dual potentials, centered shifts, log weights, and scratch buffers. The next batched update sees only surviving problems. The logical-mask control uses the same update, preliminary screen, two-marginal verifier, tolerance, and observed completion times. It freezes a passing problem's potentials but retains that problem in the batch tensor, so subsequent kernels still execute at the original width. Logical masking versus dynamic compaction therefore isolates the work removed by reducing the batch width.

Equation (2) and Equation (4) define the ideal-arithmetic transformation. Fixed-work parity, achieved residuals, near-threshold stress tests, and application endpoints characterize its implementation behavior.

Table 1: Fixed-width multi-problem execution is established in OT software. DrainSinkhorn's comparison point is the action taken after a problem satisfies its completion rule.

System	Batched numerical object	Completion signal	Execution action	Output check
POT `solve_batch` [6, 11]	Independent OT problems with batched cost matrices	Solver stopping criterion	Fixed-width parallel solve	Solver residual criterion
OTT-JAX `vmap` [4]	Vectorized independent Sinkhorn problems	Per-solve stopping or a shared iteration budget	Fixed-width vectorized execution	Solver error function
DrainSinkhorn	Positive EOT problems with compatible batched state	A preliminary screen using the current marginals, followed by the two-marginal verifier	Removes passing lanes from potentials, supports, marginals, and scratch buffers	Current row and column residual tolerance

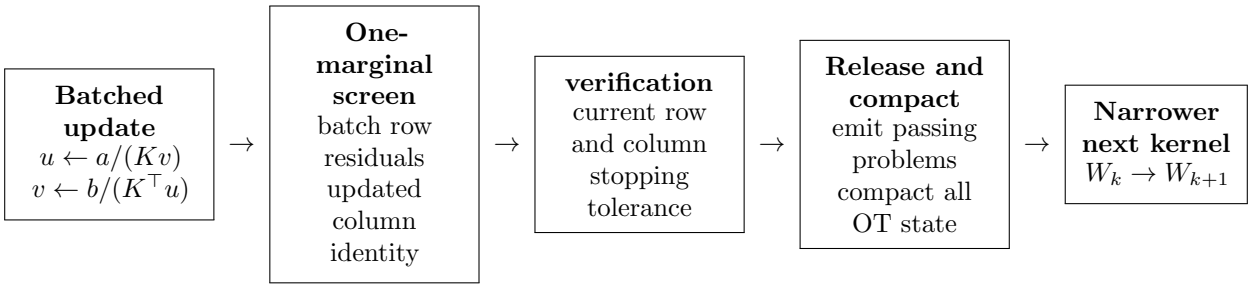

Figure 1: DrainSinkhorn's execution loop. The preliminary screen using the current marginals avoids many full plan evaluations. Problems that pass the two-marginal verifier leave every per-problem tensor; the next batched update processes only the survivors.

Stopping-rule-preserving release. A problem is removed only after passing the same two-marginal verifier used by the selected backend. The preliminary screen using the current marginals decides only whether to invoke that verification; it does not authorize removal. DrainSinkhorn therefore preserves the backend's configured release authority while changing how unfinished problems are scheduled and stored. The timed packed Triton path applies this rule in the precision supported by that backend. Independent FP64 replays serve as evaluation references for finite-precision decision behavior.

3.5 Execution modes and fallback

The implementation exposes three batch execution modes. Fixed-width batching updates every problem until a common final boundary. Logical masking records per-problem completion and freezes completed states without narrowing the kernel. Dynamic compaction removes passing problems. Width determines whether a batched kernel is efficient, and the numbers of iterations needed by the problems in the batched window determine how much width dynamic compaction can remove. Formal campaigns fix execution mode, width, verification interval, and fallback before timing. Formal campaigns use fail-closed convergence handling: iteration-limit and non-finite-state events raise an error. The packed Triton implementation optionally hands a dynamic-compaction tail to the matched single-problem solver when survivor width falls below a preconfigured `min_packed_width`. The formal MetroPT-3 and Packer19 configurations set this threshold to one.

When a complete problem fits on one accelerator, different windows are sent to independent complete-device workers, giving each device an independent update path and avoiding per-update collectives. Row sharding remains a capacity path for a single oversized problem; the main scale-out path assigns complete problems to independent devices.

4 Survivor Geometry and Hardware Conversion

The executor observes when each problem first passes the verifier. Those iteration numbers define a survivor curve, which determines removable work. The backend width-cost curve then determines how much of that work reduction is converted into wall-clock saving.

4.1 Counting removed problem-update slots

For problem t, let n_t be the batched-iteration number of the first configured verification round at which the problem passes the two-marginal verifier, and let

$$A_k = \left|\{t : n_t \geq k\}\right| \tag{6}$$

be the surviving width before packed update k. The measured verification supplies both quantities.

Proposition 1 (Problem-update accounting)**.** *For a finite window of W problems,*

$$S_{\text{active}} = \sum_{k=1}^{n_{\max}} A_k = \sum_{t=1}^{W} n_t, \qquad S_{\text{static}} = W n_{\max}. \tag{7}$$

Hence dynamic compaction removes exactly

$$R = W n_{\max} - \sum_{t=1}^{W} n_t \tag{8}$$

problem-update slots relative to a fixed-width batch with the same final boundary.

Proof. Use $n_t = \sum_{k \geq 1} \mathbf{1}\{n_t \geq k\}$ and exchange the two finite sums. Fixed-width execution keeps all W lanes through $n_{\max}$. □

Logical masking changes which states update but not the width seen by the kernel, so it retains $W n_{\max}$ slots. The area between the static rectangle and the survival curve is therefore the exact post-run work removed by dynamic compaction. The verification interval is already reflected in the observed n_t; the first passing iteration on a denser verification schedule would define a different quantity.

Proposition 2 (Task-independent opportunity condition)**.** *Let $(n_1, \ldots, n_W)$ be any random vector of finite iteration numbers. Then $\mathbb{E}[R] > 0$ if and only if there exists a batched iteration k such that*

$$\Pr(0 < A_k < W) > 0. \tag{9}$$

Proof. R is nonnegative and is zero exactly when every lane has the same iteration number. Different finite iteration numbers create a value of k with both survivors and completed lanes, and the converse follows directly from the definition of A_k. □

The condition depends on the joint distribution of iteration numbers inside a packed window. Perfectly synchronized lanes create no removable work even when the required iteration numbers vary across windows.

4.2 Exact wall-clock accounting for a measured backend

Let $c_k(a, \xi_k)$ be the cost of matched update k at width a, where ξ_k records support shape, data type, kernel configuration, and other backend state held fixed by the comparison. Then

$$\begin{aligned} T_{\text{active}} = &\sum_k c_k(A_k, \xi_k) + H_{\text{screen}} + H_{\text{verification}} \\ &+ H_{\text{compact}} + H_{\text{other}}, \end{aligned} \tag{10}$$

H_{other} covers remaining timed control work. Let H_{fixed} be the corresponding fixed-width control cost and let ΔH denote active control cost minus H_{fixed}.

Proposition 3 (Opportunity–conversion decomposition)**.** *For a matched fixed-width/dynamic-compaction execution with the same final verifier boundary and matched update states ξ_k,*

$$T_{\text{fixed}} - T_{\text{active}} = \sum_{k=1}^{n_{\max}} [c_k(W, \xi_k) - c_k(A_k, \xi_k)] - \Delta H. \tag{11}$$

Consequently, dynamic compaction is faster exactly when

$$\sum_k [c_k(W, \xi_k) - c_k(A_k, \xi_k)] > \Delta H. \tag{12}$$

Proof. Write $T_{\text{fixed}} = \sum_k c_k(W, \xi_k) + H_{\text{fixed}}$, subtract Equation (10), and collect the control terms. □

This decomposition exposes two gates. Variation among n_t creates the survivor widths needed for positive removable work. The measured function $c_k(a, \xi_k)$ determines whether those width reductions cross a lower hardware cost level. If $c_k(A_k, \xi_k) = c_k(W, \xi_k)$ for every k, dynamic compaction removes problem-update slots but saves no update time; if the summed cost decrease exceeds ΔH, it produces a net wall-clock gain.

4.3 Wave-shaped width cost

For the packed Triton update used in our implementation, the launch grid has

$$B(a, n) = a \left\lceil \frac{n}{b_m} \right\rceil \tag{13}$$

thread blocks, where b_m is the row tile size. GPU scheduling processes this grid in occupancy-dependent waves [9, 18]. A useful local model is therefore

$$c(a; n, \xi) \approx g_{n,\xi}\left(\left\lceil \frac{B(a, n)}{Q_\xi} \right\rceil \right), \tag{14}$$

where Q_ξ is the measured resident block capacity of the kernel and $g_{n,\xi}$ maps wave count to time. Registers, shared memory, and kernel state enter through Q_ξ; the value is calibrated for the selected device and kernel configuration.

On the measured A100/Triton configuration with $b_m = 64$, the first-wave estimate places the saturation width near 6.75 for $n = 1024$, 1.69 for $n = 4096$, and below one for $n = 16384$. Direct primitive measurements locate the first large increase at widths $6 \to 7$ and $1 \to 2$ for the first two sizes; the $n = 16384$ curve is responsive from width one. These measurements instantiate $c(a)$ for this backend and explain why equal slot reductions can have different wall-clock value across problem sizes.

5 Experiments

5.1 Experimental setup

We evaluate our method across four tasks: (1) **MetroPT-3** for industrial sensor streams; (2) **ImageNet-32** for flow-matching training; (3) **Packer19** for single-cell trajectory inference; and (4) **A2D2** for LiDAR point-cloud matching. These benchmark names are used throughout the paper, including the appendix.

We distinguish three timing levels. E_1 **(Sinkhorn time)** measures only the Sinkhorn solve through the verified coupling output or potentials. E_2 **(pipeline time)**

adds the fixed downstream consumer to E_1. E_3 **(total time)** measures the complete model-training, generation, or task-evaluation interval. All speedups are baseline time divided by DrainSinkhorn time, and every ratio is reported at its stated timing level. When an E_1 value is reconstructed from E_2, consumer time is subtracted within each paired timing unit before ratios, intervals, or bootstrap summaries are formed.

The main packed-Triton workloads use NVIDIA A100-SXM4-80GB GPUs, cold initialization, and the same two-marginal verifier in both arms. Matched component comparisons also hold the input, proposal, tolerance, verification schedule, and downstream consumer fixed. Complete-path comparisons use the same packed or vectorized backend in both arms. Appendix Table 8 lists the numerical settings and statistical units for each benchmark.

For ImageNet-32, every training step forms a cold-start EOT coupling between PCA500 features with $(n, W, \varepsilon) = (1024, 8, 0.03)$, verification interval 4, and stopping tolerance 10^{-3}. The paired comparison uses three seeds and 50,000 fixed training updates; endpoint quality is evaluated at NFE 4, 8, and 16. The $n = 1024, W = 8$ panel in figure 2 marks the corresponding packed-update regime, but that diagnostic excludes the rest of the E_3 training pipeline.

When stopping controllers produce different achieved residuals, we report the achieved residuals and consumer-output deviations together. The reported ratios have two causal roles:

$$\underbrace{\text{fixed-width} \to \text{dynamic}}_{\text{release and dynamic compaction}}\,,$$
$$\underbrace{\text{logical mask} \to \text{dynamic}}_{\text{width effect}}.$$

Fixed-width/dynamic-compaction measures the net change from a conventional fixed-width batch to the dynamic-compaction controller; logical-mask/dynamic-compaction (LM/DC) isolates dynamic width reduction under matched release decisions. Figure 2 reports the measured width-cost curves, and Table 2 groups registered effects by initial width. Appendix Table 10 specifies the implementation, timing scope, and statistical unit of every contrast. Bootstrap intervals on frozen workloads quantify timing reproducibility conditional on the frozen inputs and execution configuration. Each contrast is interpreted within its listed study and measurement interval.

5.2 Matched release and dynamic compaction

The matched comparisons begin from conventional fixed-width batched execution. The primary MetroPT-3 study compares the two current paths on one host with four complete-device workers. Dynamic compaction gives 1.746× on the ready-arrays-to-consumer endpoint while problem-update slots fall from 5696 to 3128. All three predeclared execution plans favor dynamic-compaction execution, with a descriptive range of 1.7453–1.7460×. The two arms each invoke 16 verifier candidates, evaluate 16 candidates, and launch 32 plan-application kernels per repeat. The corresponding four-GPU energy ratio is 1.779×.

A separate two-host MetroPT-3 study gives a fixed-width/dynamic-compaction ratio of 1.744723× on the E_2 endpoint and 1.754× at E_1 after subtracting the paired consumer within each host–plan unit. Its six host–plan units all favor dynamic-compaction execution.

The MetroPT-3 scale-out study extends the same complete-device design to eight independent workers on two hosts. Its two fixed-per-worker endpoint cells give 3.110× and 2.159× matched fixed-width/dynamic-compaction speedups while reducing slots as shown in table 3. Separate one-worker support-size cells that pass the frozen gates on both hosts give 2.316× at $n = 8192$ and 1.616× at $n = 32768$.

In the ImageNet-32 training study, each step uses a real PCA500 feature coupling. Table 9 reports the four solver-phase times. Paired solver-field fixed-width/dynamic-compaction is 1.054× (observed seed range 1.033–1.072×; table 11). On the complete fixed-update training interval, fixed-width/dynamic-compaction is 1.036× (range 1.030–1.043×). A separate ImageNet-32 diagnostic measures coupling preparation over three problem–plan seeds, with fixed-width/dynamic-compaction 1.105×. The study that uses one initialization for multiple problems supplies a non-cold setting: across 40 controlled overlapping-support windows, compacted execution is 1.421× faster, reduces slots by 30.62%, and wins every paired window.

The numerical quality results use the same three paired training seeds at NFE 4, 8, and 16. The full per-seed measurements and analysis artifacts are available in the public GitHub repository at `https://github.com/cis-aconiticacid/DrainSinkhorn`. The evaluation measures the ImageNet-32 PCA500 feature-space OT-FM endpoint.

5.3 Measured conversion from width to kernel time

We measure the width-cost term in equation (11) by calling the same packed Triton update at fixed problem size and varying only the width. Figure 2 normalizes the measured medians as $g_W(a) = c(W, \xi)/c(a, \xi)$ for static widths $W = 8$ and 16, using the common survivor-width coordinates $a \in \{1, 2, 3, 4, 6, 8, 12, 16\}$. For a realized survivor width A_k, the corresponding summand in equation (11) is $c(W, \xi_k)[1 - g_W(A_k)^{-1}]$. The panel value $\Delta = b_m N_{\mathrm{SM}}/n$ predicts the plateau scale: conversion is quantized at $n = 1024$ ($\Delta = 6.75$), becomes finer at $n = 4096$ ($\Delta = 1.69$), and is near continuous at $n = 16384$ ($\Delta = 0.42$).

The diagnostic isolates the packed update, so it estimates one component of $c_k(a, \xi_k)$ rather than the complete executor. Its $n = 1024, W = 8$ shape is consistent with

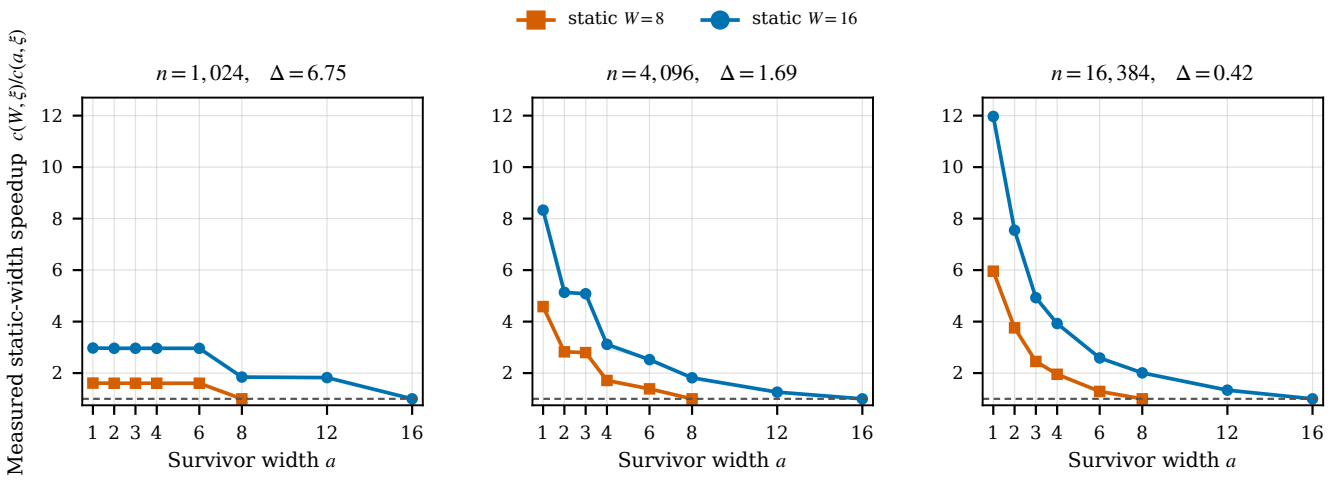

Figure 2: Measured conversion from survivor width to packed-update speedup. Each curve reports $g_W(a) = c(W,\xi)/c(a,\xi)$ relative to static width $W \in \{8,16\}$; all panels use the same axes, and shading propagates the per-width interquartile endpoints from 30 repeats. Panel titles report $\Delta = b_m N_{\mathrm{SM}}/n$, which determines scheduling-wave quantization. The $n = 1024, W = 8$ setting matches the ImageNet-32 training solver configuration; the plotted measurements time only the packed update, not total training.

Table 2: Registered matched results grouped by initial width. Speedup is baseline time divided by dynamic-compaction time. Brackets give registered 95% intervals and parentheses give observed seed or plan ranges. Rows retain their study-specific timing scopes and are not pooled. Row-level provenance is recorded in `support/figure_materials/MAIN_RESULTS_BY_WIDTH.csv`.

W	Campaign	Configuration	Contrast	Speedup	Timing scope and statistical unit
8	ImageNet-32	$n = 1024$	fixed/DC	1.054× (1.033–1.072)	solver field; 3 training seeds
8	ImageNet-32	$n = 1024$	fixed/DC	1.105× (1.079–1.156)	coupling preparation; 3 problem–plan seeds
8	Packer19	$n = 16384$	LM/DC	1.5397× [1.5389, 1.5404]	consumer-subtracted OT stage; 24 paired units
8	Packer19	$n = 16384$	LM/DC	1.5713× [1.5701, 1.5726]	direct OT stage at 10^{-3}; 24 paired units
8	MetroPT-3	$n = 16384$, route 3	fixed/DC	2.159×	eight-worker complete endpoint; fixed work per worker
8	MetroPT-3	$n = 16384$, route 2	fixed/DC	3.110×	eight-worker complete endpoint; fixed work per worker
16	ImageNet-32	$n = 4096$, shared anchor	fixed/DC	1.421×	steady-state critical path; 40 windows
16	ImageNet-32	$n = 4096$, OTT-JAX	fixed/DC	1.581×	matched 20-update phases; 10 repeats
16	MetroPT-3	$n = 32768$, route 3	fixed/DC	1.616×	one-worker endpoint; equal-host geometric mean
16	MetroPT-3	$n = 16384$, two hosts	fixed/DC	1.7447× [1.7434, 1.7467]	ready-arrays-to-consumer endpoint; 6 host–plan units
16	MetroPT-3	$n = 16384$, four workers	fixed/DC	1.7455× (1.7453–1.7460)	four-worker makespan; 3 registered plans
16	MetroPT-3	$n = 8192$, route 3	fixed/DC	2.316×	one-worker endpoint; equal-host geometric mean

Table 3: MetroPT-3 fixed-per-worker execution on eight complete-device workers across two hosts. Times aggregate the complete consumer endpoint. Slots count problem updates across workers; energy uses the same endpoint.

Route	Fixed (s)	Dynamic (s)	Slots fixed/dynamic	Time	Energy
failure-2	12.523	4.026	4736 / 1276	3.110×	3.437×
failure-3	7.999	3.705	3008 / 1244	2.159×	2.292×

the modest ImageNet-32 solver-field ratio: after the survivor width enters the $A \le 6$ region, further lane removal changes slot count more than packed-update time. The $n = 16384$ response provides the corresponding hardware condition for the larger MetroPT-3 dynamic-compaction gains; the matched endpoint then measures the complete net effect, including control and consumer work.

Transfer across backend families. The OTT-JAX and PyKeOps rows test the execution principle with independent implementations. OTT-JAX execution that processes only the remaining problems in each fixed phase is 2.600× faster than native OTT-JAX on real ImageNet-32 at $W = 16$; its matched fixed-width/dynamic-compaction control assigns 1.545× to removing completed problems only at phase boundaries and then rebucketing the sur-

Table 4: ImageNet-32 PCA500 feature-space quality after 50k training updates. Entries are three-seed means for matched fixed-width (Fixed) and dynamically compacted (Dynamic) execution. Lower is better for all three metrics. The same three paired training seeds are used throughout the quality evaluation; the full per-seed measurements and analysis artifacts are available in the public GitHub repository. These are feature-space metrics, not pixel-space FID.

	Curvature		Held-out FM MSE		Projected Fréchet	
NFE	Fixed	Dynamic	Fixed	Dynamic	Fixed	Dynamic
4	5.4524	5.4448	1.07854	1.07859	1.2967	1.2886
8	2.8388	2.8356	1.07854	1.07859	1.5286	1.5224
16	1.4546	1.4528	1.07854	1.07859	1.8182	1.8136

vivors. On real A2D2 LiDAR, the PyKeOps compacted path is 3.174× faster than sequential carry at $W = 8$. A real ImageNet-32 PyKeOps cell remains positive at 1.415×. The tiled-log implementation reaches $n = 65536$ without materializing per-problem cost tensors and supplies the measured capacity boundary.

Independent OTT-JAX and PyKeOps realizations are summarized in table 7.

5.4 Packer19 single-variable component tests

The two-host Packer19 stopping study evaluates residual tolerances 10^{-4}, 3×10^{-4}, 10^{-3}, 3×10^{-3}, and 10^{-2} without aggregating across tolerances. All arms use the same two-marginal verifier and residual tolerance, and the same tight-reference consumer output.

Dynamic compaction improves over logical masking by 1.480–1.638× at 10^{-4} through 3×10^{-3}. At 10^{-2}, all eight problems first pass at update 8, so there is no removable padding and LM/DC is neutral at 0.9995× (95% CI [0.9987, 1.0003]). These contrasts characterize Packer19's complete and logical-mask paths; phase costs are reported separately in tables 5 and 12.

The single-variable tests use the same Packer19 transition workload on two hosts, each with four A100-SXM4-80GB GPUs. Three frozen method orders produce 24 paired host–order–GPU units. Each unit uses warmup-excluded repeat medians, the same input, packed-kernel source tree, 10^{-3} stopping tolerance, and transition consumer. As shown in Table 5, each row changes one execution component under this shared measurement boundary.

Table 5: Packer19 single-variable attribution under one frozen input and release contract. Every row reports OT-stage time from a paired comparison after subtracting consumer time, so the component effects share one measurement boundary.

Change	Matched contrast	Ratio and evidence
Batched verification	serial / batched verifier	1.133×; 24/24 wins
Current-marginal screen	verifier only / preliminary screen	1.105×; 24/24 wins
Logical freeze	fixed / logical mask	1.000×; 95% CI [0.9998, 1.0003]
Dynamic compaction	logical mask / dynamic compaction	1.540×; 95% CI 1.5389–1.5404×

The batched-verification and current-marginal-screen rows have 24/24 paired wins; no separate E_1 bootstrap interval is registered for those two contrasts.

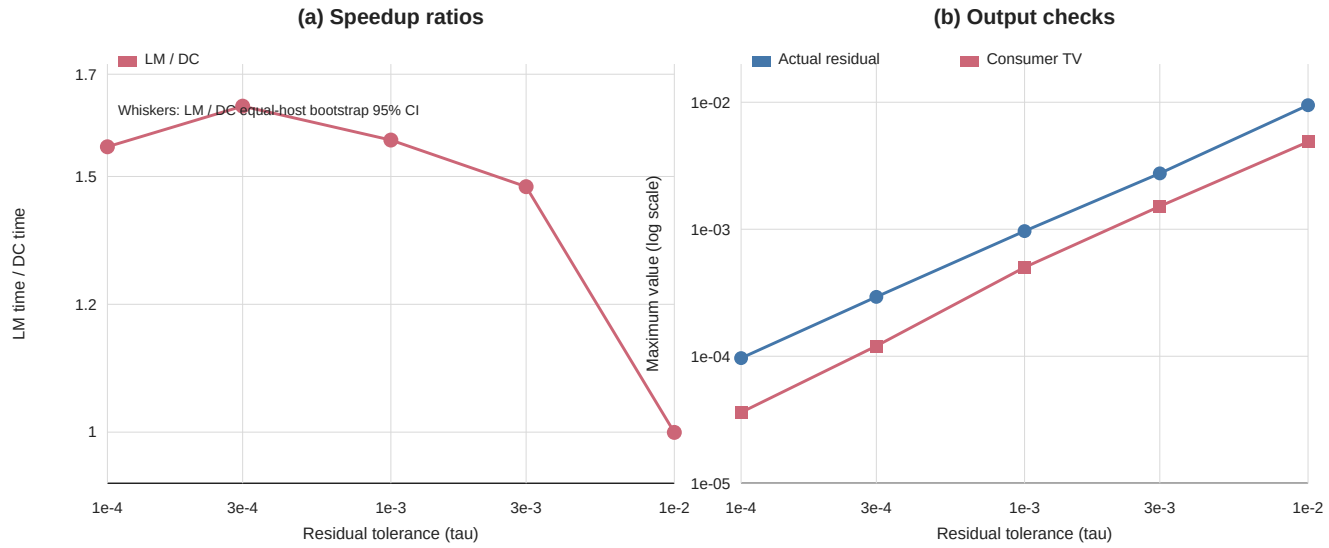

Figure 3: Two-host Packer19 stopping–execution results under a common two-marginal verifier and residual tolerance. Panel (a) shows the LM/DC speedup ratio across the five tolerances. Whiskers are the equal-host stratified bootstrap 95% intervals for LM/DC. Panel (b) shows maximum actual residual and maximum consumer TV on a logarithmic scale. Each point summarizes 24 paired units; consumer TV measures differences in outputs against the tight reference.

Verifier and screen ablation The controls expose distinct work. Batched evaluation of the two-marginal verifier and the preliminary screen using the current marginals reduce OT-stage time under matched paths. Each ratio subtracts its paired consumer time before forming the contrast.

We replay all 1488 screen-negative events and evaluate each with the two-marginal verifier, finding 0 already-releasable lanes. The maximum difference between the preliminary screen and verifier row residual is 1.86×10^{-7}. Every emitted plan passes the two-marginal verifier at 10^{-3}.

Logical masking versus dynamic compaction. Logical masking is neutral because the packed kernel retains width eight. Dynamic compaction reduces the observed width, removes 100 of 256 problem-update slots, and yields 1.540× OT-stage acceleration with interval 1.5389–1.5404×. All 24 paired units favor dynamic compaction. Across the 24 host–order–GPU unit medians, endpoint energy falls from 879.9 to 688.1 J, while compacted buffers raise peak allocated memory from 80.8 to 106.9 MiB. All registered endpoint quality checks pass.

5.5 MetroPT-3 grouping test

The grouping test holds a set of 32 real MetroPT-3 problems fixed and changes only their assignment to windows. Offline every-cycle iteration numbers q_t define the grouping, while verifier-derived iteration numbers n_t measure executed work. Oracle homogeneous grouping minimizes variation in the required iteration numbers within each window. Heterogeneous and random groupings increase the area between the static rectangle and the survival curve defined by n_t. As shown in Table 6, the matched fixed-width/dynamic-compaction ratio grows from 1.105× to 1.217× and 1.235–1.271× on random layouts.

Table 6: MetroPT-3 grouping intervention on one fixed real task multiset. For offline every-cycle iteration numbers q_s within a window, $H_q = \mathrm{sd}(q_s)/\,\mathrm{mean}(q_s)$ and padding is $W \max_s q_s / \sum_s q_s - 1$. Dynamic slots is the ratio of dynamic to static candidate-iteration slots. The displayed values aggregate over windows; timing repeats stabilize each condition and are not independent datasets.

Grouping	H_q	Padding	Dynamic slots	Fixed/dynamic
heterogeneous	0.236	0.236	0.802	1.217×
homogeneous	0.127	0.110	0.892	1.105×
random 1	0.254	0.293	0.776	1.258×
random 2	0.264	0.270	0.789	1.235×
random 3	0.260	0.279	0.764	1.271×

5.6 Numerical safety and scale-out

Independent finite-precision stress test. An independent stress test measures decision behavior on 84 near-threshold problems and 16 runtime attacks over 1/2/4/8 GPUs. Relative to independent FP64 replay, raw FP32 and TF32 each disagree on three accept decisions; an outward-bound guard followed by FP64 escalation has zero disagreements in these 100 cases. This guarded configuration is evaluated separately from the timed applications.

Timed applications and independent-worker scale-out. The timed packed Triton path uses the precision supported by the backend and the two-marginal verifier. All emitted plans meet both recorded marginal-residual bounds, and every endpoint passes its registered output or quality check. The MetroPT-3 scale-out study assigns complete windows to independent GPU workers. Four workers achieve 3.965–3.979× fixed-per-worker throughput scaling on held-out MetroPT routes, with no Sinkhorn collective between workers; the eight-worker matched endpoint decomposition appears in table 3.

6 Operating Regime and Composition

DrainSinkhorn applies to batched positive EOT problems with compatible batched representations and a two-marginal verifier. Different numbers of iterations within a batch create removable work; net acceleration depends on runtime as a function of batch width and on incremental control and state-movement costs (equation (12)).

Initialization shapes both the total iteration count and its variation within a window. The study that uses one initialization for multiple problems uses controlled overlapping ImageNet-32 supports: dynamic-compaction execution reduces slots by 30.62% and accelerates the steady-state critical path by 1.421×, including proposal and transfer costs. The feature-training study uses cold proposals for independently resampled OT-FM couplings.

The executor schedules problems from completion under the configured stopping rule. The grouping intervention measures the effect of grouping a fixed problem multiset using offline oracle iteration numbers. The width-cost sweep separately measures how the backend converts each survivor width into update time. Together they instantiate the two terms in equation (11); predictive grouping can use the same quantities when estimates of completion depth are available.

Dynamic compaction trades computation for state movement and buffers. The matched logical-mask/dynamic-compaction study reports 26.1 MiB of additional peak allocation alongside lower time and endpoint energy; table 12 reports its phase costs. OTT-JAX and PyKeOps realize release through backend-specific rebucketing and batched independent-problem execution.

The calibrated width-cost curve belongs to a specified device, kernel path, problem shape, and numerical configuration. The packed Triton path also switches narrow tails to its matched single-problem solver, so its complete cost curve includes that routing boundary. The accounting identities and equation (11) apply to any measured cost curve; the observed A100 wave locations apply to the reported Triton configuration.

7 Conclusion

DrainSinkhorn removes padded OT work from batches whose problems require different numbers of iterations. It starts from standard fixed-width execution of independent Sinkhorn problems; the selected solver's stopping rule determines completion, and dynamic compaction narrows every subsequent per-problem kernel. The survival-curve identity counts the problem updates that disappear. The opportunity–conversion decomposition then maps survivor width through the measured backend cost curve and subtracts control overhead, giving the exact matched-path break-even condition.

Matched fixed-width/dynamic-compaction comparisons deliver 1.746× on the four-worker MetroPT-3 endpoint, 1.054× on the ImageNet-32 training solver field, and 1.616–3.110× in the registered MetroPT scale cells. A non-cold setting that uses one initialization for multiple problems gives 1.421× with 30.62% fewer problem-update slots. Independent OTT-JAX and PyKeOps implementations remove completed problems through independent backends.

Matched controls identify the mechanism. Batched verification and the Sinkhorn-specific screen reduce inspection cost; logical masking leaves the kernel width unchanged; dynamic compaction removes OT work. In the registered MetroPT and Packer19 endpoint windows, dynamic compaction also reduces measured GPU energy. Grouping while holding total iteration counts fixed shows that greater variation in the required iteration numbers within a window creates more removable padding. Direct A100 width sweeps show a wave-shaped conversion curve: small widths can share one cost level, while larger problems respond to the width reduction earlier. width reduction earlier. The reported applications pair speed with achieved residuals and consumer or training endpoints. Fast inner kernels make each update cheaper; DrainSinkhorn stops issuing those updates to completed problems.

Reproducibility. The release artifact records source and input identities, commands, software and GPU environments, raw outputs, numerical checks, and paired analyses.

References

[1] Jason Altschuler, Jonathan Niles-Weed, and Philippe Rigollet. Near-linear time approximation algorithms for optimal transport via Sinkhorn iteration. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL `https://papers.nips.cc/paper_files/paper/2017/hash/491442df5f88c6aa018e86dac21d3606-Abstract.html`.

[2] Brandon Amos, Giulia Luise, Samuel Cohen, and Ievgen Redko. Meta optimal transport. In *Proceedings of the 40th International Conference on Machine Learning*, volume

202 of *Proceedings of Machine Learning Research*, pages 791–813, 2023. URL https://proceedings.mlr.press/v202/amos23a.html.

[3] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transportation distances. In *Advances in Neural Information Processing Systems*, volume 26, pages 2292–2300, 2013. URL https://arxiv.org/abs/1306.0895.

[4] Marco Cuturi, Laetitia Meng-Papaxanthos, Yingtao Tian, Charlotte Bunne, Geoff Davis, and Olivier Teboul. Optimal transport tools (OTT): A JAX toolbox for all things wasserstein, 2022. URL https://arxiv.org/abs/2201.12324.

[5] Jean Feydy, Thibault Séjourné, François-Xavier Vialard, Shun ichi Amari, Alain Trouvé, and Gabriel Peyré. Interpolating between optimal transport and MMD using sinkhorn divergences. In *Proceedings of the 22nd International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pages 2681–2690, 2019. URL https://proceedings.mlr.press/v89/feydy19a.html.

[6] Rémi Flamary, Nicolas Courty, Alexandre Gramfort, Mokhtar Z. Alaya, Aurélie Boisbunon, Stanislas Chambon, Laetitia Chapel, Adrien Corenflos, Kilian Fatras, Nemo Fournier, Léo Gautheron, Nathalie T. H. Gayraud, Hicham Janati, Alain Rakotomamonjy, Ievgen Redko, Antoine Rolet, Antony Schutz, Vivien Seguy, Danica J. Sutherland, Romain Tavenard, Alexander Tong, and Titouan Vayer. POT: Python optimal transport. *Journal of Machine Learning Research*, 22(78):1–8, 2021. URL https://www.jmlr.org/papers/v22/20-451.html.

[7] Ginkgo Project. Batched CG. Ginkgo user guide, 2026. URL https://gko-project.org/user-guide/concepts/batched/cg.html. Accessed 2026-09-08.

[8] Mete Kemertas, Amir massoud Farahmand, and Allan D. Jepson. A truncated newton method for optimal transport, 2025. URL https://arxiv.org/abs/2504.02067.

[9] *CUDA C++ Programming Guide*. NVIDIA, 2024. URL https://docs.nvidia.com/cuda/archive/12.5.0/cuda-c-programming-guide/index.html.

[10] Aram-Alexandre Pooladian, Heli Ben-Hamu, Carles Domingo-Enrich, Brandon Amos, Yaron Lipman, and Ricky T. Q. Chen. Multisample flow matching: Straightening flows with minibatch couplings. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 28100–28127, 2023. URL https://proceedings.mlr.press/v202/pooladian23a.html.

[11] POT developers. POT 0.9.6 release notes. Python Optimal Transport documentation, 2025. URL https://pythonot.github.io/releases.html.

[12] Alexey Radul, Brian Patton, Dougal Maclaurin, Matthew D. Hoffman, and Rif A. Saurous. Automatically batching control-intensive programs for modern accelerators. In *Proceedings of Machine Learning and Systems*, volume 2, 2020. URL https://proceedings.mlsys.org/paper_files/paper/2020/hash/39945d578f616735572174bf5e8f155d-Abstract.html.

[13] Meyer Scetbon, Marco Cuturi, and Gabriel Peyré. Low-rank Sinkhorn factorization. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 9344–9354, 2021. URL https://proceedings.mlr.press/v139/scetbon21a.html.

[14] Alexis Thibault, Lénaïc Chizat, Charles Dossal, and Nicolas Papadakis. Overrelaxed Sinkhorn–Knopp algorithm for regularized optimal transport, 2017. URL https://arxiv.org/abs/1711.01851.

[15] James Thornton and Marco Cuturi. Rethinking initialization of the sinkhorn algorithm. In *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics*, volume 206 of *Proceedings of Machine Learning Research*, pages 8682–8698, 2023. URL https://proceedings.mlr.press/v206/thornton23a.html.

[16] John Viljoen, Johanna Haffner, Masayoshi Tomizuka, and Negar Mehr. Scaling nonlinear optimization: Many problems one GPU, 2026. URL https://arxiv.org/abs/2606.26341.

[17] Felix X.-F. Ye, Xingjie Li, An Yu, Ming-Ching Chang, Linsong Chu, and Davis Wertheimer. FlashSinkhorn: IO-aware entropic optimal transport on GPU, 2026. URL https://arxiv.org/abs/2602.03067.

[18] Geoffrey X. Yu, Yubo Gao, Pavel Golikov, and Gennady Pekhimenko. Habitat: A runtime-based computational performance predictor for deep neural network training. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pages 503–521, 2021. URL https://www.usenix.org/conference/atc21/presentation/yu.

[19] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation*, pages 521–538, 2022. URL https://www.usenix.org/conference/osdi22/presentation/yu.

[20] Stephen Zhang, Alireza Mousavi-Hosseini, Michal Klein, and Marco Cuturi. On fitting flow models with large sinkhorn couplings, 2025. URL https://arxiv.org/abs/2506.05526.

A Measurement Contracts and Additional Detail

A.1 ImageNet-32 seed provenance

The quality evaluation uses the same three paired training seeds, 20260891, 20260892, and 20260893, throughout the reported NFE values. The full per-seed measurements and analysis artifacts are available in the public GitHub repository at https://github.com/cis-aconiticacid/DrainSinkhorn.

Measurement levels. E1 terminates at the measured coupling output or potentials after the two-marginal verifier passes. E2 includes the fixed downstream consumer. E3 includes training, generation, or task evaluation. The MetroPT-3 and Packer19 pipeline studies record E_2 endpoints, whereas ImageNet-32 records an E_3 training endpoint. The primary MetroPT-3 ratio uses its registered E_2 endpoint directly. The two-host MetroPT-3 and Packer19 component studies subtract each unit's paired consumer time before ratio formation and bootstrap, while ImageNet-32 uses the recorded solver field. The Packer19 stopping study records E_1 directly for every stopping-controller and execution-mode cell. Unchanged consumer or model time in E_2 or E_3 dilutes the corresponding OT-stage gain. The ImageNet-32 fixed-width/dynamic-compaction ratio reported as a training ablation is computed from the complete fixed-update E_3 interval.

GPU-energy interval. GPU energy is the change in the A100 NVML cumulative total-energy counter from immediately before the registered method call to after the final CUDA synchronization. The counter reports millijoules and the analysis converts the difference to joules. This ready-arrays-to-consumer interval includes solver/proposal and consumer work and excludes compilation, warm-up, and the prepared-input transfer component. Unsupported counters are recorded as unavailable rather than replaced by a power-limit estimate.

Primary MetroPT-3 matched fixed/dynamic control. This study runs fixed-width and dynamic-compaction paths on the same MetroPT-3 failure-3 input, cold proposal, regularization, tolerance, verification schedule, two-marginal verifier, packed backend, and fixed consumer. The experimental variable is whether verifier-approved lanes are dynamically removed from later kernels. One host supplies four independent complete-device A100 workers, with no inter-worker collective or problem sharding. For each plan and timing repeat, the endpoint is the maximum ready-arrays-to-consumer time across the four workers. Each plan is summarized by the median of two timing repeats, and the reported 1.745535× is the descriptive geometric mean of the three plan ratios. The plan identifiers freeze execution order and are not independent MetroPT samples. All 48 worker-level repeats converge; the maximum row and column L_1 residuals are 9.476650×10^{-4} and 1.283915×10^{-6}. Both arms attain failure-window ROC AUC 1.0 under the frozen consumer, whose labels are not used by the solver. The largest paired transport-cost and barycentric-shift differences are 0.02792263 and 0.00346756.

Two-host MetroPT-3 matched control. Within this matched contrast, fixed-width and dynamic-compaction execution use the same inputs, cold proposal, regularization, tolerance, verification schedule, two-marginal verifier, packed backend, and fixed consumer; stopping-rule-triggered release and dynamic compaction are the controlled difference. Three order-balanced plan seeds are reused across four GPU placements on each of two hosts. GPU placements and timing repeats stabilize end-to-end measurement; the registered endpoint ratio includes unchanged non-OT consumer time. The main-text MetroPT-3 OT-stage ratio subtracts the paired consumer time within each host–plan unit. The host–plan combinations are paired timing units for this frozen workload; placements and repeats are timing replicas rather than independent MetroPT workloads. A reanalysis of the six valid host–plan units reports a stratified bootstrap interval.

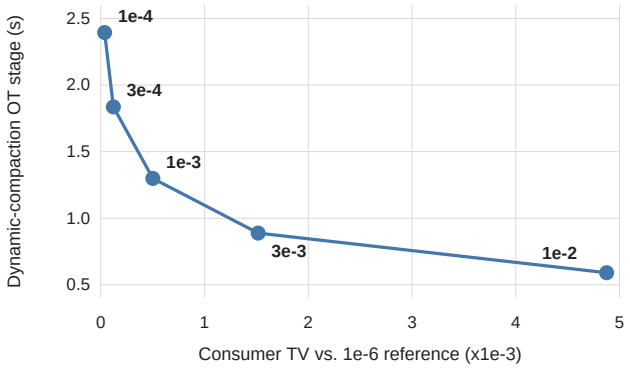

Figure 4: Tolerance-dependent Packer19 OT speed–output frontier for the dynamic-compaction path. Each point is the median E1 OT-stage time over 24 paired units at that residual tolerance; consumer TV is measured against the campaign's 10^{-6} tight reference.

Packer19 stopping-study provenance. The stopping–execution protocol has SHA-256 `c010bcf9b3e276c8269514324cc8ed4bc2fc5958c362c7b880e3946dee75075b` and is bound to source commit `96de52b8ceff8fa3c0468bf45567e6b017483e73`. Two hosts, four A100-SXM4-80GB GPUs per host, three frozen method orders, and three timing repeats produce 768 formal cells. Each tolerance has 24 paired host–order–GPU units and an equal-host bootstrap interval. Consumer TV numerically checks differences in outputs.

Execution-mode protocol. The formal campaigns fix representation, width, execution mode, verification interval, stopping tolerance, verification path, tail-handoff threshold, and fail-closed policy before timing. The width and grouping interventions map conditions for selecting fixed-width execution or dynamic compaction.

Width-cost diagnostic provenance. The primitive sweep uses one NVIDIA A100-SXM4-80GB with 108 SMs, driver 535.129.03, PyTorch 2.5.1 with CUDA 12.4, Triton

3.1.0, and source commit `73aec2941bfa1a5caa6a6bddf7aa83b734f9bb8b`. Each measured call to `shifted_step` contains two packed Triton log-sum-exp kernels and excludes the marginal screen, two-marginal verifier, dynamic compaction, and outer `solve_batch` loop. Widths are randomized within 30-repeat blocks after warm-up. The $n = 1024$ sweep covers widths 1–8, 10, 12, and 16; the $n = 4096$ and $n = 16384$ sweeps cover 1, 2, 3, 4, 6, 8, 12, and 16. This diagnostic supplies the packed-update component of the measured width-cost curve in equation (11).

Independent backend realizations. The OTT-JAX and PyKeOps paths remove completed problems through their native backend mechanisms. OTT-JAX executes fixed 20-update phases, verifies at phase boundaries, removes passing lanes, and rebuckets survivors; native `vmap` and a matched fixed-phase static path provide its controls. The PyKeOps paths use one batched independent-problem matrix-free operator, a fixed geometric verification schedule, the two-marginal verifier, and removal of passing problem state. Sequential and static-padded PyKeOps paths use the same symbolic backend. The shared principle is per-problem release followed by dynamic removal; verifying and packing mechanisms remain backend-specific. All cross-backend rows in table 7 report their achieved two-marginal residual.

A.2 Independent backend implementations

B Deployment and Operating Boundaries

Complete-device execution and partitioning the matrix by rows. When one problem fits on one accelerator, complete-device workers process different windows and require no Sinkhorn collective. Partitioning the matrix by rows supplies a capacity path for an oversized single problem. Complete-device execution is the throughput path used by the MetroPT-3 scale-out study. On one host, increasing from one to four workers yields 3.965–3.979× fixed-per-worker throughput scaling on two held-out MetroPT routes. The two-host, eight-worker fixed-per-worker run reports endpoint comparisons at that worker count. We report the one-to-four and two-host eight-worker results as distinct scaling studies. Peak memory remains local to each worker.

B.1 Supplementary attribution and evidence boundaries

The ImageNet-32 coupling-preparation diagnostic holds the packed Triton adapter, input, and numerical contract fixed and changes dynamic compaction; its three fixed-width/dynamic-compaction ratios are 1.156, 1.082, and 1.079. Fixed-width/dynamic-compaction slots are 64/52, 64/56, and 64/56. The measurements cover coupling preparation. The OTT-JAX study uses one frozen ImageNet-32 input and ten timing repeats per tolerance; the phase size is chosen on a disjoint warmup bank. Its 0.995× result at 10^{-2} is retained alongside the positive matched-phase results.

The MetroPT-3 global eight-worker results aggregate the complete consumer endpoint at fixed work per worker. Fixed/dynamic seconds are 12.523/4.026 on route 2 and 7.999/3.705 on route 3, with slots 4736/1276 and 3008/1244. The reported support-size speedups comprise the route-3 cells at $n = 8192$ and 32768 that pass the frozen gates on both hosts. Continuous transport-cost and barycentric-shift differences are diagnostics under the registered binary consumer endpoint.

The ImageNet-32 shared-anchor result uses a constructed 90%-overlap target stream over real ImageNet features. Its canonical reverse-order warm replay excludes compile, cache load and process startup, while retaining descriptor, transfer, anchor, proposal, correction and verification costs. The MetroPT-3 grouping ratios describe one fixed 32-problem multiset; oracle iteration numbers q_t define the grouping intervention. The fixed multiset fixes $\sum_t q_t$, while grouping changes $\sum_w W q_{\max,w}$ across windows. The ratios report the resulting relative-runtime effect; absolute arm times under each grouping would separate movement of the fixed-width and dynamic-compaction paths.

Table 7: Independent OTT-JAX and PyKeOps realizations on real data. Every ratio is baseline time divided by the compacted path, and every uncertainty label states whether it is a repeat or observed-input range.

Backend	Workload	OT-stage contrast	Speedup (uncertainty)	Timing; unit	Accuracy contract
OTT-JAX	ImageNet-32, $n = 4096, W = 16$	native vmap / remaining problems processed in each fixed phase	2.600× (repeat range 2.338–2.664×)	direct E1; 6 timing repeats	max L1 9.97×10^{-4}
PyKeOps	A2D2 LiDAR, $n = 8192, W = 8$	sequential carry / active	3.174× (clip range 2.180–3.944×)	direct E1; 3 real clips	max L1 9.97×10^{-4}
PyKeOps	ImageNet-32, $n = 16384, W = 4$	sequential / active	1.415× (seed range 1.360–1.448×)	direct E1; 3 seeds	max L1 8.93×10^{-4}

Table 8: Configuration of the main formal workloads. All rows use cold initializations and a verifier that checks both current marginal residuals.

Benchmark	Endpoint, numerical configuration, and statistical unit
MetroPT-3, four-worker	Predictive maintenance (E_2); $(n, W, \varepsilon) = (16{,}384, 16, .1)$; verification interval 4; $\tau = 10^{-3}$; one host, four complete-device workers, three order plans, two timing repeats.
MetroPT-3, two-host	Predictive maintenance (E_2); $(n, W, \varepsilon) = (16{,}384, 16, .1)$; verification interval 4; $\tau = 10^{-3}$; two hosts and three plans.
Packer19 components	Single-cell transition (E_2); $(16{,}384, 8, .1)$; verification interval 4; $\tau = 10^{-3}$; 24 paired units.
Packer19 stopping	Same-controller Pareto (E_1); $(16{,}384, 8, .1)$; verification interval 4; $\tau = 10^{-4}$–10^{-2}; 24 paired units per tolerance.
ImageNet-32	PCA500 OT-FM training (E_3); $(1{,}024, 8, .03)$; verification interval 4; $\tau = 10^{-3}$; three paired seeds and 50,000 fixed updates.

Table 9: Absolute OT-stage time (seconds) for the three packed-Triton workloads at the 10^{-3} operating point. Packer19 reports the median over 24 paired host–order–GPU units. ImageNet-32 reports solver-phase mean ± sample standard deviation over three paired training seeds. MetroPT-3 reports the median of the three registered worker-1 plan units. Packer19 has no reported fixed-width value because its registered fixed-width arm executed the compaction branch and is excluded. A dash marks an unavailable matched arm. These absolute summaries use the listed units; paired effects are reported separately.

Workload	Fixed-width batch	Logical mask	Dynamic compaction
Packer19 ($n = 16384, W = 8$)	—	2.038	1.299
MetroPT-3 ($n = 16384, W = 16$)	14.839	—	8.462
ImageNet-32 PCA500	122.17 ± 2.91	—	115.89 ± 0.83

Table 10: Evidence map for the reported attribution contrasts. Ratios are numerator-path time/denominator-path time; values above one favor the denominator. GM denotes geometric mean. Timing scope and sample/replication units are listed by study. The shared-anchor ImageNet-32 row is an older project-adapter study on controlled overlapping supports. Regularization is 0.03 for the PCA500 ImageNet-32 rows and 0.1 for the other rows. The source ledger retains the full study inventory and exclusions.

Study	Backend / workload	$n, W; \tau$	Proposal	Contrast	Ratio	Timing scope	Sample / replication
MetroPT-3 four-worker	packed Triton / MetroPT-3	$16384, 16; 10^{-3}$	cold	fixed/ dynamic	1.745535	E_2 four-worker makespan; slots 5696/3128 = 1.820972×	plans 1.745273 / 1.745356 / 1.745975; 2 repeats
MetroPT-3 two-host	packed Triton / MetroPT-3	$16384, 16; 10^{-3}$	cold	fixed/ dynamic	1.754	E_1; paired consumer subtracted	host–plan units
ImageNet-32 solver	packed Triton / ImageNet-32	$1024, 8; 10^{-3}$	cold	fixed/ dynamic	1.054	solver field; supplementary GM	3 training seeds
ImageNet-32 training	packed Triton / ImageNet-32	$1024, 8; 10^{-3}$	cold	fixed/ dynamic	1.036	complete fixed-update training	3 training seeds
ImageNet-32 preparation	packed Triton / ImageNet-32	$1024, 8; 10^{-3}$	cold	fixed/ dynamic	1.105	coupling preparation	3 problem–plan seeds
MetroPT-3 eight-worker	packed Triton / MetroPT routes 2,3	$16384, 8; 10^{-3}$	cold	fixed/ dynamic	3.110 / 2.159	8-worker endpoint; fixed per worker	2 routes; host/plan repeats
MetroPT-3 support size	packed Triton / MetroPT route 3	$8192, 16; 10^{-3}$	cold	fixed/ dynamic	2.316	one-worker endpoint; equal-host GM	2 hosts; 3 plans each
MetroPT-3 support size	packed Triton / MetroPT route 3	$32768, 16; 10^{-3}$	cold	fixed/ dynamic	1.616	one-worker endpoint; equal-host GM	2 hosts; 3 plans each
ImageNet-32 OTT-JAX	OTT-JAX / ImageNet-32	$4096, 16; 10^{-3}$	fixed	fixed/ dynamic	1.581	matched 20-update phases	one input; 10 repeats
ImageNet-32 OTT-JAX	OTT-JAX / ImageNet-32	$4096, 16; 3\times10^{-3}$	fixed	fixed/ dynamic	1.556	matched 10-update phases	one input; 10 repeats
ImageNet-32 OTT-JAX	OTT-JAX / ImageNet-32	$4096, 16; 5\times10^{-3}$	fixed	fixed/ dynamic	1.366	matched 10-update phases	one input; 10 repeats
ImageNet-32 OTT-JAX	OTT-JAX / ImageNet-32	$4096, 16; 10^{-2}$	fixed	fixed/ dynamic	0.995	matched 10-update phases	one input; 10 repeats
ImageNet-32 shared anchor	packed Triton / ImageNet-32	$4096, 16; 10^{-3}$	shared anchor	fixed/ dynamic	1.421	steady-state critical path	40 controlled windows
Packer19 fixed/ mask	packed Triton / Packer19	$16384, 8; 10^{-3}$	cold	fixed/LM	1.000	E_1; paired consumer subtracted	24 host–order–GPU units
Packer19 mask/ compact	packed Triton / Packer19	$16384, 8; 10^{-3}$	cold	LM/DC	1.540	E_1; paired consumer subtracted	24 host–order–GPU units
Packer19 tolerance sweep	packed Triton / Packer19	$16384, 8; 10^{-4}$–3×10^{-3}	cold	LM/DC	1.480–1.638	direct E_1; four tolerances	24 paired units/tolerance
Packer19 synchronized release	packed Triton / Packer19	$16384, 8; 10^{-2}$	cold	LM/DC	0.9995	direct E_1; synchronized release	24 paired units/tolerance

Table 11: ImageNet-32 solver seconds and fixed/dynamic by seed. Descriptive paired reanalysis of existing records; the geometric mean of the three ratios is 1.054.

Seed	Fixed (s)	Dynamic (s)	Fixed/dynamic
20260891	123.429	116.699	1.058
20260892	124.233	115.934	1.072
20260893	118.840	115.036	1.033

Table 12: Packer19 phase times (seconds): marginal medians for each component of the matched logical-mask and dynamic-compaction implementations. Each phase is summarized separately; total-time ratios are computed from paired whole-path timings and reported in table 5.

Phase	LM (s)	PC (s)
Correction	1.7063	1.0708
Batched verification	0.6385	0.4006
Verifier	0.1045	0.1045
Mask / compaction	0.000306	0.003370